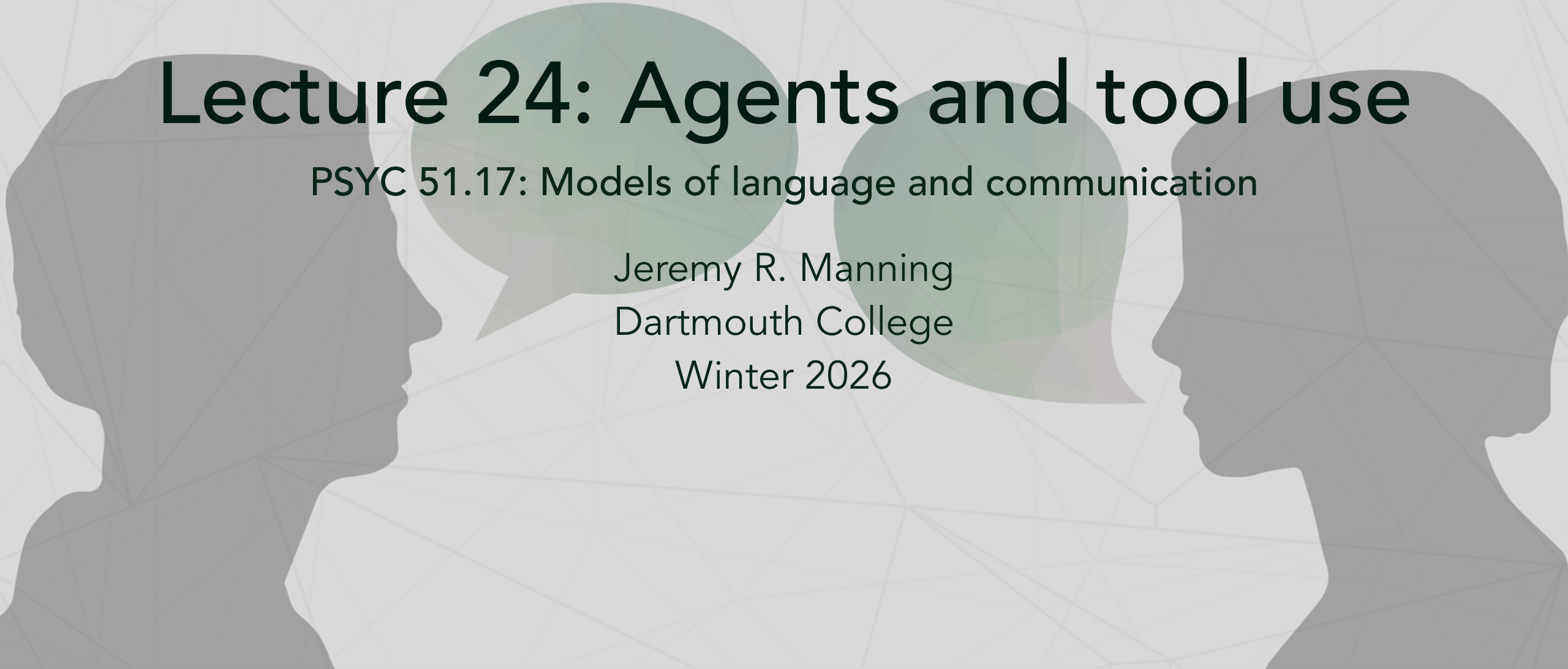
Lecture 24: Agents and tool use

PSYC 51.17: Models of language and communication

Jeremy R. Manning
Dartmouth College
Winter 2026



Learning objectives

By the end of this lecture, you will be able to...

- Define what an **LLM agent** is and how it differs from a chatbot
- Explain **function calling** and how LLMs interact with external tools
- Describe the **ReAct framework** (Yao et al., 2023) and its reasoning-action loop
- Explain how **Toolformer** (Schick et al., 2023) teaches models to use tools autonomously
- Implement a **simple agent loop** with tool dispatch in Python
- Discuss the opportunities and risks of **autonomous AI agents**

From chatbots to agents

What is an LLM agent?

An **agent** is a system where a language model can:

1. **Reason** about a task (plan steps, reflect on progress)
2. **Act** by calling external tools (search, code execution, APIs)
3. **Observe** the results of those actions
4. **Iterate** until the task is complete

A chatbot *responds to prompts*. An agent *takes actions in the world*.

Questions to consider

When you use ChatGPT to browse the web or run Python code, you are interacting with an agent. What makes this qualitatively different from a simple question-answering system?

Why tools matter

LLMs are powerful but limited

Language models trained on text alone cannot:

- **Access current information** (training data has a cutoff date)
- **Perform precise computation** (arithmetic, symbolic math)
- **Interact with external systems** (databases, APIs, file systems)
- **Verify their own claims** (no ground truth access)

Tools compensate for LLM weaknesses

By connecting an LLM to external tools, we can combine the model's **language understanding and reasoning** with tools that provide **accuracy, recency, and real-world interaction**. The LLM decides *what* to do; the tools *do* it.

Function calling

How LLMs use tools

Function calling is a mechanism where the LLM outputs a structured request to invoke an external function, rather than generating free-form text. The system executes the function, and the result is fed back to the LLM.

Function calling with the OpenAI API

```
tools = [{
    "type": "function",
    "function": {
        "name": "get_weather",
        "description": "Get current weather for a location",
        "parameters": {
            "type": "object",
            "properties": {
                "location": {"type": "string", "description": "City
name"},
                "unit": {"type": "string", "enum": ["celsius",
```

Function calling

Function calling with the OpenAI API

```
            },
            "required": ["location"]
        }
    }
}]
```

...continued

Function calling in practice

The LLM decides when and how to call tools

```
# User asks: "What's the weather like in Hanover, NH?"

# Step 1: LLM generates a function call (not free text)
response = {
    "function_call": {
        "name": "get_weather",
        "arguments": '{"location": "Hanover, NH", "unit": "fahrenheit"}'
    }
}

# Step 2: System executes the function
result = get_weather(location="Hanover, NH", unit="fahrenheit")
# Returns: {"temperature": 28, "condition": "snowy"}

# Step 3: Result is fed back to the LLM
```

The ReAct framework

Yao et al. (2023): 'ReAct: Synergizing Reasoning and Acting'

ReAct interleaves two capabilities in a loop:

- **Reasoning** (chain-of-thought): The model thinks about what to do next
- **Acting**: The model calls a tool or takes an action
- **Observing**: The model reads the result and decides the next step

This continues until the task is complete or the model decides it has enough information.

Why interleaving matters

Pure reasoning (chain-of-thought alone) can hallucinate facts. Pure acting (tool use without reasoning) can call the wrong tools. **ReAct combines both**: the model reasons about *which* tool to use, acts, then reasons about the result.

ReAct in action

Answering a question with search

```
Question: "What is the elevation of the city where Dartmouth College
          is located?"

Thought 1: I need to find which city Dartmouth College is in.
Action 1:  search("Dartmouth College location")
Obs 1:     Dartmouth College is in Hanover, New Hampshire.

Thought 2: Now I need the elevation of Hanover, NH.
Action 2:  search("Hanover New Hampshire elevation")
Obs 2:     Hanover, NH has an elevation of 531 feet (162 m).

Thought 3: I have the answer.
Action 3:  finish("531 feet (162 meters)")
```

Each **Thought** is the model reasoning; each **Action** is a tool call; each **Obs** is the tool's response.

ReAct vs. chain-of-thought

Comparison of reasoning approaches

Approach	Reasoning	Tool use	Strengths	Weaknesses
Standard prompting	None	None	Simple	No reasoning, no grounding
Chain-of-thought	Yes	None	Better reasoning	Can hallucinate facts
Act-only	None	Yes	Grounded in tool results	May call wrong tools
ReAct	**Yes**	**Yes**	**Grounded reasoning**	More complex, more tokens

Empirical results (Yao et al., 2023)

On knowledge-intensive tasks (HotpotQA, FEVER), ReAct outperformed chain-of-thought by **reducing hallucinations** through grounded search. On reasoning tasks (ALFWorld, WebShop), ReAct outperformed act-only methods by making **more informed tool choices**.

Toolformer: self-taught tool use

Schick et al. (2023): 'Toolformer: Language Models Can Teach Themselves to Use Tools'

Toolformer trains an LLM to decide *when and how* to call tools by embedding API calls directly into training text. The model learns to insert tool calls at positions where they reduce prediction loss.

How Toolformer annotates training data

```
Original:  "The Eiffel Tower is 330 meters tall."
Annotated: "The Eiffel Tower is [QA("height of Eiffel Tower") → 330]
            meters tall."

Original:  "The population of France is about 67 million."
Annotated: "The population of France is about
            [Search("France population") → 67 million] ."
```

The model learns to insert `[Tool(query) → result]` at positions where the tool call improves next-token prediction. At inference time, the system intercepts these

Common agent tools

Tools available to modern LLM agents

Tool	Purpose	Example
Web search	Access current information	"What happened in the news today?"
Code interpreter	Execute Python, do math	"Calculate the eigenvalues of this matrix"
File system	Read/write files	"Save this analysis to report.csv"
Database	Query structured data	"How many users signed up last month?"
API calls	Interact with services	"Send an email to the team"
Browser	Navigate web pages	"Fill out this form and submit it"
Image generation	Create visual content	"Draw a diagram of this architecture"

Implementing a simple agent loop

A minimal agent in Python

```python
import openai, json

TOOLS = {
    "search": lambda q: web_search(q),
    "calculate": lambda expr: eval(expr),  # Simplified!
    "finish": lambda answer: answer,
}

def agent_loop(question, max_steps=5):
    messages = [{"role": "user", "content": question}]
    for step in range(max_steps):
        response = openai.chat.completions.create(
```

continued...

Implementing a simple agent loop

A minimal agent in Python

```
            model="gpt-4", messages=messages,
tools=tool_definitions
        )
        msg = response.choices[0].message
        if msg.tool_calls:
            for call in msg.tool_calls:
                fn = TOOLS[call.function.name]
                result = fn(**json.loads(call.function.arguments))
                messages.append({"role": "tool", "content":
str(result),
                                 "tool_call_id": call.id})
        else:
            return msg.content  # Final answer (no tool call)
    return "Max steps reached"
```

...continued

The agent loop pattern

The universal agent architecture

Nearly all LLM agents follow the same core loop:

1. **Prompt** the LLM with the task and available tools
2. **Parse** the LLM's output for tool calls or a final answer
3. **Execute** any requested tool calls
4. **Append** tool results to the conversation
5. **Repeat** until the LLM produces a final answer (or a step limit is reached)

Critical design decisions

- **Maximum steps**: Prevents infinite loops (typically 5--20)
- **Error handling**: What happens when a tool call fails?
- **Sandboxing**: Code execution must be isolated for safety
- **Cost control**: Each loop iteration costs API tokens

Multi-step reasoning example

An agent solving a complex task

```
User: "Compare the GDP per capita of the US and Japan in 2023,
       and calculate the ratio."

Step 1 - Thought: I need GDP per capita data for both countries.
Step 1 - Action:  search("US GDP per capita 2023")
Step 1 - Result:  "$76,329"

Step 2 - Action:  search("Japan GDP per capita 2023")
Step 2 - Result:  "$33,950"

Step 3 - Thought: Now I can calculate the ratio.
Step 3 - Action:  calculate("76329 / 33950")
Step 3 - Result:  2.248

Step 4 - Answer:  "US GDP per capita ($76,329) is approximately
                   2.25x Japan's ($33,950) in 2023."
```

The agent **decomposed** the task, **gathered data** with search, **computed** the ratio

Autonomous agents

Agents that operate with minimal human oversight

Autonomous agents extend the basic agent loop to handle long-running, complex tasks with multiple sub-goals. Key additional capabilities include:

- **Planning**: Break complex goals into sub-tasks
- **Memory**: Maintain context across many steps (beyond context window)
- **Self-reflection**: Evaluate and correct their own work
- **Delegation**: Spawn sub-agents for parallel tasks

Notable autonomous agent systems

- **AutoGPT** (2023): Goal-driven agent that creates and manages sub-tasks
- **BabyAGI** (2023): Task-driven autonomous agent with prioritized task list
- **Voyager** (2023): Minecraft agent that writes code to acquire new skills
- **SWE-Agent** (2024): Autonomously fixes GitHub issues by writing code

Planning and decomposition

How agents handle complex goals

Complex tasks require the agent to **plan** before acting. Common planning strategies:

Task decomposition: Break a goal into ordered sub-tasks

```
Goal: "Write a research report on climate change impacts in NH"
→ Sub-task 1: Search for recent climate data for New Hampshire
→ Sub-task 2: Find specific impact studies (agriculture, tourism, wildlife)
→ Sub-task 3: Synthesize findings into a structured report
→ Sub-task 4: Add citations and format
```

Reflection and replanning: After each step, evaluate progress and adjust the plan if needed.

Questions to consider

Human experts naturally decompose problems, monitor progress, and adjust plans.

Agent memory

Overcoming the context window limitation

LLM context windows are finite (4K--128K tokens). Long-running agents need additional memory:

Memory type	Implementation	Use case
Short-term	Conversation history in context	Recent steps and observations
Working memory	Scratchpad / notepad tool	Intermediate results, running totals
Long-term	Vector database (RAG)	Past experiences, learned procedures
Episodic	Structured logs	What worked/failed in previous runs

The memory bottleneck

Memory management is one of the hardest problems in agent design. Too little context and the agent forgets its plan. Too much context and it becomes slow, expensive, and confused by irrelevant information.

Safety and control

Risks of agentic AI systems

- **Unintended actions**: An agent told to "clean up my email" might delete important messages
- **Goal misalignment**: An agent optimizing a metric might find harmful shortcuts
- **Cascading errors**: One bad tool call can lead to a chain of incorrect actions
- **Resource exhaustion**: Autonomous agents can run up large API bills
- **Security**: Agents with code execution can potentially access sensitive systems

Mitigation strategies

- **Human-in-the-loop**: Require approval for irreversible actions
- **Sandboxing**: Run code in isolated environments with no network access
- **Budget limits**: Cap API calls, compute time, and token usage
- **Audit logging**: Record every action for review
- **Capability restrictions**: Only grant tools the agent actually needs

The broader landscape

How agents fit into the LLM ecosystem

Paradigm	LLM role	Example
Chatbot	Respond to queries	ChatGPT (basic mode)
RAG system	Answer with retrieved context	Lecture 17's pipeline
Tool-using LLM	Call functions on demand	ChatGPT with plugins
ReAct agent	Reason + act iteratively	Research assistant
Autonomous agent	Plan + execute complex goals	SWE-Agent, Devin
Multi-agent system	Multiple LLMs collaborating	Debate, code review

Questions to consider

We have moved from models that *generate text* (GPT-1) to models that *take actions* (agents). What are the implications of this shift for how we think about AI alignment and safety?

Key takeaways

Core concepts from this lecture

1. **Agents** extend LLMs beyond text generation by adding reasoning, tool use, and iterative action
2. **Function calling** lets LLMs generate structured tool requests that systems execute
3. **ReAct** (Yao et al., 2023) interleaves reasoning and acting for grounded problem-solving
4. **Toolformer** (Schick et al., 2023) trains models to decide when and how to use tools autonomously
5. The **agent loop** (prompt → parse → execute → observe → repeat) is the universal pattern
6. **Safety** requires human oversight, sandboxing, budgets, and careful capability restrictions

Further reading

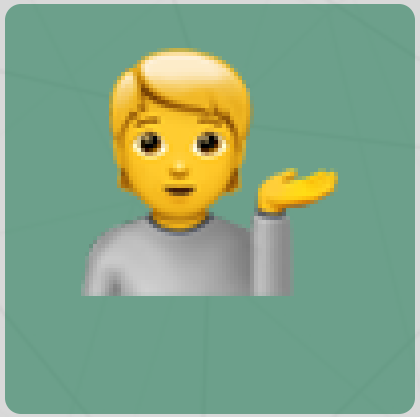
References

- **Yao et al. (2023)** -- "ReAct: Synergizing Reasoning and Acting in Language Models" [arXiv]
- **Schick et al. (2023)** -- "Toolformer: Language Models Can Teach Themselves to Use Tools" [arXiv]
- **Wei et al. (2022)** -- "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models" [arXiv]
- **Wang et al. (2024)** -- "A Survey on Large Language Model based Autonomous Agents" [arXiv]
- **OpenAI Function Calling** -- [Documentation]

Questions?

Email

Discord



Office hours

Up next...

Mixture of experts and efficiency: scaling models without scaling compute